OralSight

Oral lesion screening and review prototype | Ralph + Person 2 | Python + React

Core scope: image upload/capture, quality feedback, lesion classification, uncertainty, Grad-CAM, evaluation dashboard and downloadable summary. Academic prototype; no cancer staging or treatment advice.

Owner	Responsibility	Planned effort
Ralph (you)	ML, dataset, evaluation and research documentation	40 hours
Person 2	React UI, FastAPI, integration and deployment	40 hours
Both	Milestone reviews, final testing and demonstration	Included above

Schedule and checkpoints

When	Ralph	Person 2	Checkpoint
Days 1-2	Audit data; define labels and splits	Repository, API contract and UI skeleton	Dataset accepted; contract agreed
Days 3-5	Baseline and fine-tuning	Upload, quality UI and API routes	First end-to-end prediction
Days 6-8	Calibration, heatmaps and model package	Results, evidence dashboard and export	Feature-complete local app
Days 9-10	Final metrics and robustness analysis	Mobile, API and deployment tests	Model and feature freeze
Days 11-12	Research report and evidence slides	Setup guide, screenshots and demo video	Submission-ready build
Days 13-14	Rehearsal and fixes	Rehearsal and fixes	Buffer; no new major features

Dataset gate - finish before model training

Confirm access, licence, original-image/patient counts, class labels, duplicates and patient IDs. The original Karnataka dataset has **323 images**; treat it as a pilot if no stronger dataset is secured. A larger dataset is an unresolved dependency, not an assumed resource. [1]

Validation rule: split by patient where possible; keep duplicates together; augment training images only. If patient IDs are unavailable, disclose that limitation. Do not add healthy/precancerous labels without corresponding data.

Time basis: 80 planned person-hours + 16-24 hours shared contingency = 96-104 total. Allow 12-14 days at roughly 3-4 hours per person/day. Compute and download waits are additional elapsed time.

Ralph: ML + research

Primary owner of data quality, model behaviour and the evidence behind every result.

ID / window	Task	Deliverable / handoff	Hours
R1 / D1-2	Audit dataset, licence, classes, patient IDs and duplicates	Dataset audit + approved label mapping to Person 2	5
R2 / D2-3	Build manifest, grouped splits, preprocessing and train-only augmentation	Frozen split files + preprocessing configuration	4
R3 / D3-4	Train frozen image embeddings + logistic regression baseline	Baseline metrics and reproducible script	3
R4 / D4-5	Fine-tune MobileNetV3 or EfficientNet-B0; save best validation checkpoint	Model weights + class order + inference function	6
R5 / D6	Define blur/exposure checks; calibrate scores and abstention threshold on validation data	Quality rules + score/abstention configuration	3
R6 / D6-7	Implement Grad-CAM and inspect representative outputs	Explanation function; heatmap caveat	3
R7 / D8-10	Evaluate locked model, confidence intervals, errors, calibration and image corruptions	metrics.json, plots and reviewed failure cases	6
R8 / D8-10	Package model; verify train/inference consistency and CPU timing	Versioned model bundle; integration support	3
R9 / D11-12	Write data/method/results/limitations; prepare evidence slides	Research report sections + model card	5
R10 / D13-14	Review final system; rehearse model explanation and viva	Signed-off metrics and demo cases	2

Tools

Python: PyTorch, torchvision, pandas, NumPy, Pillow, OpenCV, scikit-learn, Matplotlib/Seaborn, Captum or pytorch-grad-cam. **Training:** Colab; CPU baseline fallback.

Required analysis

Confusion matrix; malignant-class recall; specificity; F1; ROC-AUC/PR-AUC; confidence intervals; calibration; coverage versus error under abstention; blur/compression/exposure robustness; baseline comparison; CPU latency. Analyse clinical subgroups only when metadata exists.

Done when: model loads on Person 2's laptop, preprocessing matches training, test data stayed untouched during selection, and every chart uses saved actual results. Heatmaps are model attention, not verified lesion boundaries.

Person 2: app + backend

Primary owner of the complete user flow, integration, mobile compatibility and reliable demonstration.

ID / window	Task	Deliverable / handoff	Hours
P1 / D1-2	Create repository, React/Vite + FastAPI setup; agree API contract	Runnable skeleton + clearly labelled mock response	3
P2 / D2-3	Build responsive upload/capture, preview, crop and retry flow	Phone-friendly input page	5
P3 / D3-4	Implement image validation, size limits, EXIF orientation and errors	Safe upload endpoint; clear unsupported-format message	4
P4 / D4-6	Integrate Ralph's preprocessing, quality rules and model; load once at startup	Working /health and /analyze endpoints	5
P5 / D6-7	Build scores, abstention, quality feedback and image/heatmap toggle	Complete result page	5
P6 / D7-8	Render saved metrics and model comparison; add print-to-PDF summary	Evidence page + export	4
P7 / D8-9	Test phone/browser, API failures, corrupted files and repeated requests	Bug log + essential regression checks	4
P8 / D9-10	Package local build; optional public hosting; benchmark CPU and startup	Reproducible startup + local fallback	4
P9 / D11-12	Write setup/architecture sections; capture screenshots and backup video	README + report sections + 60-90 sec video	4
P10 / D13-14	Rehearse upload, results and failure recovery with Ralph	Ready-to-run demo folder	2

Tools

Frontend: React, TypeScript, Vite, Tailwind CSS, Recharts, browser print-to-PDF. **Backend:** Python, FastAPI, Uvicorn, Pydantic, python-multipart. **Collaboration:** Git/GitHub.

API and integration contract

Response fields: predicted label, class scores, quality status/reasons, abstention flag, heatmap availability, model version and latency. Ralph owns prediction semantics; Person 2 owns API delivery and UI states.

Done when: upload to export works with real inference; model loads once; invalid input gives clear feedback; images are not retained by default. No database/accounts in the MVP. Use HTTPS for live browser camera access; keep file upload as fallback. [4]

Resources + demo

Keep the local build reliable; treat public hosting and extra features as optional.

Resource	Plan	Owner / constraint
Training	Colab free; save checkpoints after useful runs	Ralph: GPU access is not guaranteed. [2]
Demo runtime	Existing laptop + phone; local CPU inference	Person 2: test actual hardware and phone browser
Public hosting	Optional Vercel frontend + Render backend	Person 2: check eligibility/memory; Render sleeps after 15 min and restarts in about 1 min. [3,5]
Storage / costs	Session-only images; no paid APIs or database	Both: zero-cost route uses local runtime; cloud limits apply
Performance budget	Target <3 sec prediction; 5-10 sec with explanation	Both: targets, not measured claims; benchmark before demo
Compute allowance	Initial run: roughly 15-60 min; experiments: several hours	Ralph: estimates vary with dataset and hardware

Five-minute presentation split

Time	Presenter	Action
0:00-0:30	Ralph	Problem, dataset and exact prediction scope
0:30-1:10	Person 2	Phone upload/capture + quality feedback
1:10-2:10	Person 2	Held-out image: prediction, heatmap and export
2:10-2:50	Ralph	Uncertain/failed example; explain model limits
2:50-3:50	Ralph	Real metrics, baseline comparison and robustness
3:50-4:30	Person 2	Architecture, runtime and local fallback
4:30-5:00	Both	Contributions, impact and next steps

Final handoff checklist

Ralph: model weights, split manifests, training scripts, model card, metrics and research sections.
Person 2: React/FastAPI code, dependency files, setup guide, screenshots and demo recording.
Both: final report, slides, source/licence details, plagiarism report per brief and rehearsal.

Demo pack: benign, malignant, uncertain/misclassified and degraded examples; all clearly labelled. Use held-out data for classification, not healthy teammates as benign/malignant test cases. Keep app, weights and examples offline. Add segmentation only after the core build works and suitable masks exist.

Source links / operational references

1. Original oral dataset (323 originals) | 2. Colab availability and limits | 3. Render free-service limitations | 4. Browser camera security requirements | 5. Vercel Hobby eligibility